

Lista de exercícios — Árvores AVL

Estrutura base sugerida em C:

```
typedef struct NoArv {  
    int valor;  
    struct NoArv* esq;  
    struct NoArv* dir;  
    int altura;  
} NoArv;
```

Observações:

- Nesta lista, considere que uma árvore AVL é uma árvore binária de busca balanceada, isto é, para todo nó, o fator de balanceamento deve estar entre -1 e 1 .
- Quando for pedido para desenhar a árvore, represente claramente os filhos esquerdo e direito.
- Nas questões de implementação, considere que não há valores repetidos.
- Quando houver remoção de um nó com dois filhos, utilize o **sucessor em ordem**, salvo indicação em contrário.

1. Em uma árvore AVL, assinale a alternativa correta.

- (a) Toda árvore AVL é apenas uma árvore binária qualquer, sem restrições de ordem.
- (b) Toda árvore AVL é uma ABB em que a diferença entre as alturas das subárvores esquerda e direita da raiz é, 0 , -1 ou 1 .
- (c) Toda árvore AVL precisa ser completa.
- (d) Em uma árvore AVL, o fator de balanceamento de qualquer nó deve ser 0 , 1 ou -1 .

2. Em uma árvore AVL, o **fator de balanceamento** de um nó é dado por:

- (a) altura da subárvore direita menos altura da subárvore esquerda.
- (b) quantidade de filhos à esquerda menos quantidade de filhos à direita.
- (c) altura da subárvore esquerda menos altura da subárvore direita.

(d) quantidade total de descendentes do nó.

3. Classifique cada afirmação em **V** ou **F**.

I. Toda árvore AVL é uma ABB válida.

II. Uma árvore AVL pode ficar temporariamente desbalanceada durante uma inserção, mas ao final da operação deve voltar a satisfazer as restrições de altura.

III. Se o fator de balanceamento de um nó é 2, então esse nó está balanceado.

IV. Em uma árvore AVL, a travessia em ordem produz os valores em ordem crescente.

V. Uma árvore com apenas um nó é uma AVL válida.

4. Assinale a alternativa correta sobre rotações em árvores AVL.

(a) A rotação simples à direita é usada quando o fator de balanceamento é menor que -1.

(b) A rotação simples à esquerda é usada quando o fator de desbalanceamento é menor ou igual a -1.

(c) A rotação dupla esquerda-direita é aplicada quando o desbalanceamento ocorre para a esquerda, mas a inserção foi feita na subárvore direita do filho esquerdo.

(d) Rotações servem apenas para modificar a ordem dos valores da árvore.

5. Faça as seguintes inserções e remoções em uma árvore AVL, originalmente vazia.

Inserir:

24, 66, 71, 13, 17, 23, 49, 31, 37, 82, 11

Remover:

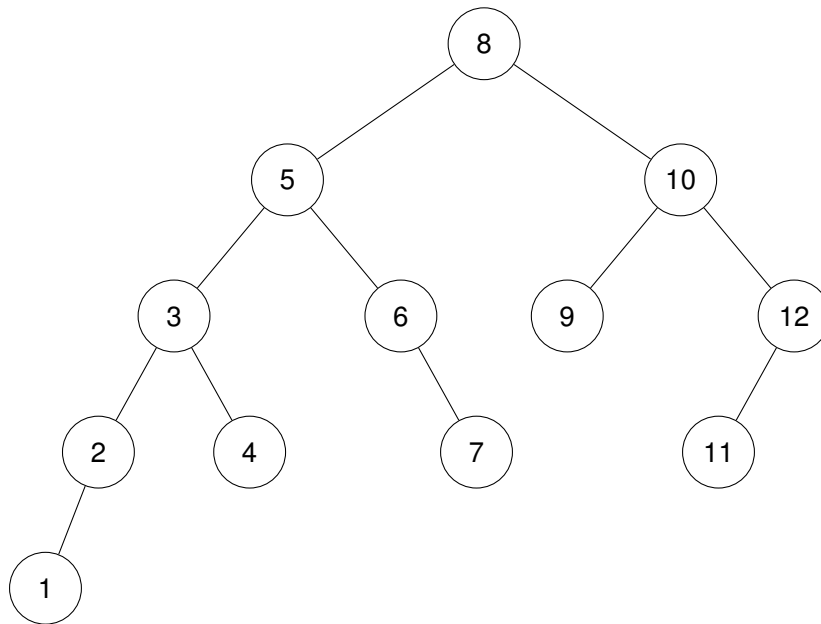
82, 13, 23, 49

Desenhe a árvore após **todas** as operações, respeitando o balanceamento AVL.

6. Faça um esquema gráfico que represente a estrutura final de uma árvore AVL gerada pela inserção dos valores abaixo, **nesta ordem**:

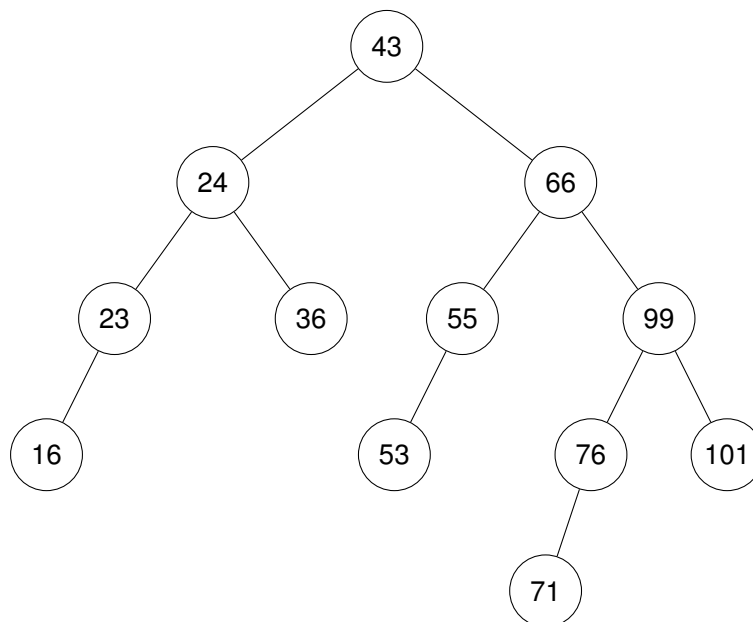
13, 7, 31, 43, 8, 17, 5, 3, 1, 23, 16, 36

7. Considere a árvore AVL abaixo.



Remova o nó 8 e, em seguida, insira o valor 9. Desenhe a árvore AVL resultante após as duas operações.

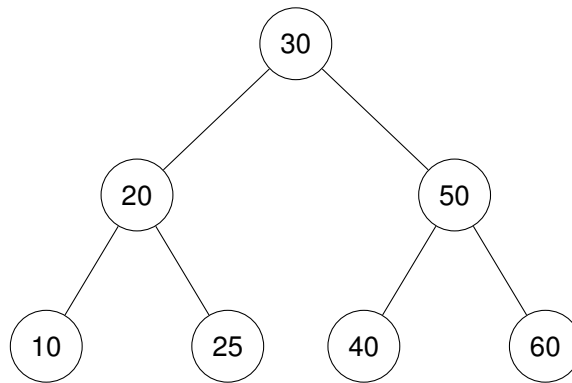
8. Considere a árvore AVL abaixo.



Execute as operações abaixo, **uma de cada vez**, desenhando a árvore resultante após **cada** operação:

Inserir 107, Inserir 20, Remover 99, Remover 36

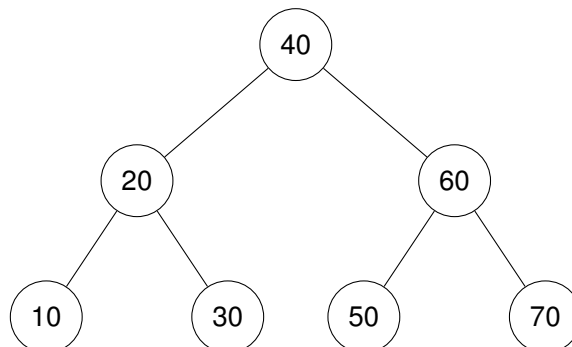
9. Dada a AVL abaixo:



Execute, **sempre a partir da árvore original**, cada uma das operações abaixo. Em cada caso, desenhe a árvore resultante e indique a rotação realizada, quando houver.

- (a) insira o valor 70;
- (b) insira o valor 45;
- (c) remova o valor 10;
- (d) remova o valor 60.

10. Considere a árvore AVL abaixo.



Assinale a alternativa correta.

- (a) A inserção do valor 80 exigirá uma **rotação simples à esquerda**.
- (b) A remoção do valor 10 exigirá uma **rotação dupla direita-esquerda**.
- (c) A inserção do valor 25 **não exigirá nenhuma rotação**.
- (d) A remoção do valor 70 exigirá uma **rotação simples à direita**.
- (e) A inserção do valor 55 exigirá uma **rotação dupla esquerda-direita**.

11. Implemente uma função que percorra uma AVL e conte quantos nós possuem **dois filhos**.

```
int contarNosComDoisFilhos(NoArv* raiz);
```

12. Implemente uma função que retorne **1** se a árvore estiver perfeitamente balanceada na raiz, isto é, com fator de balanceamento igual a 0, e **0** caso contrário.

```
int raizPerfeitamenteBalanceada(NoArv* raiz);
```

13. Um parque de diversões organiza seus brinquedos por código em uma AVL para localizar rapidamente uma atração específica. Escreva uma função que imprima o caminho da raiz até o valor procurado, caso ele exista.

```
int mostraCaminhoAVL(NoArv* raiz, int valor);
```

A função pode retornar 1 quando encontra o valor e 0 caso contrário.

14. Considere o trecho abaixo, retirado da implementação estudada em aula:

```
NoArv* balancear(NoArv* raiz) {
    int fb = fatorBalanceamento(raiz);

    if (fb > 1) {
        if (fatorBalanceamento(raiz->esq) >= 0) {
            return rotacaoDireita(raiz);
        } else {
            return rotacaoEsquerdaDireita(raiz);
        }
    } else if (fb < -1) {
        if (fatorBalanceamento(raiz->dir) <= 0) {
            return rotacaoEsquerda(raiz);
        } else {
            return rotacaoDireitaEsquerda(raiz);
        }
    }
    return raiz;
}
```

Explique o que aconteceria se, no caso $fb > 1$ e $fatorBalanceamento(raiz->esq) < 0$, o programador trocasse `rotacaoEsquerdaDireita(raiz)` por `rotacaoDireita(raiz)`.

15. Analise o trecho abaixo.

```
NoArv* inserirRecBalanceada(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRecBalanceada(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRecBalanceada(raiz->dir, valor);
    }

    raiz->altura = maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));
    raiz = balancear(raiz);
    return raiz;
}
```

Há um erro lógico na atualização da altura. Identifique-o e reescreva a linha correta.

16. Analise o trecho abaixo.

```
int fatorBalanceamento(NoArv* no) {
    if (no == NULL) {
        return 0;
    }
    return alturaDoNo(no->dir) - alturaDoNo(no->esq);
}
```

Considerando a convenção usada nas rotações vistas em aula, explique por que esse trecho pode causar decisões erradas no balanceamento.

17. Analise o trecho abaixo.

```
NoArv* removerRecBalanceada(NoArv* raiz, int valor) {
    if (raiz == NULL) return NULL;

    if (valor < raiz->valor)
        raiz->esq = removerRecBalanceada(raiz->esq, valor);
    else if (valor > raiz->valor)
        raiz->dir = removerRecBalanceada(raiz->dir, valor);
    else {
        if (raiz->esq == NULL || raiz->dir == NULL) {
            NoArv* temp = (raiz->esq != NULL) ? raiz->esq : raiz->dir;
            free(raiz);
            return temp;
        }

        raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));
        raiz = balancear(raiz);
        return raiz;
    }
}
```

Explique por que essa versão está incompleta para o caso de remoção de nó com **dois filhos**.

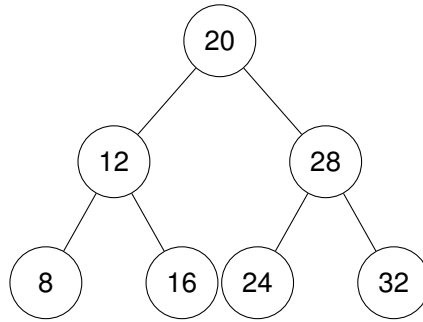
18. Considere a sequência de inserções em uma AVL inicialmente vazia:

48, 26, 12, 37, 61, 55, 74, 52, 58

Responda:

- (a) Qual será a raiz da árvore ao final?
- (b) Quem será o pai do nó 58?
- (c) O nó 37 ficará como filho esquerdo ou direito de qual nó?
- (d) A árvore final exigiu rotação simples, dupla, ambas ou nenhuma?

19. Considere a árvore AVL abaixo.



Classifique cada afirmação em **V** ou **F**.

- I. Ao inserir o valor 4, a árvore continuará AVL sem necessidade de rotação.
- II. Ao inserir o valor 2 após a inserção de 4, poderá ocorrer um caso esquerda-esquerda.
- III. Ao remover o nó 32 da árvore original, nenhuma rotação será necessária.
- IV. Ao remover 32 e depois 28, a raiz da árvore necessariamente mudará.

20. Uma rede de bibliotecas organiza os livros raros por código em uma AVL. Para manutenção do sistema, deseja-se identificar rapidamente situações específicas. Implemente as funções abaixo:

- (a) uma função que conte quantas folhas existem na AVL;
- (b) uma função que retorne o menor valor armazenado;
- (c) uma função que retorne o maior valor armazenado.

```

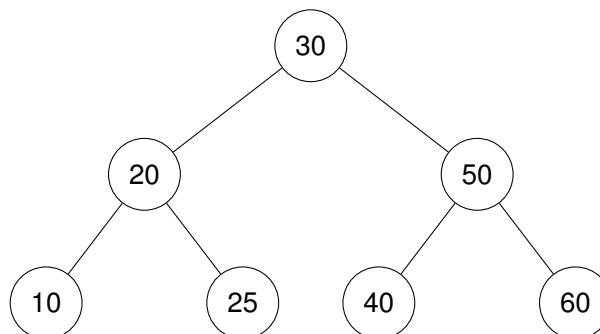
int contarFolhasAVL(NoArv* raiz);
int menorAVL(NoArv* raiz);
int maiorAVL(NoArv* raiz);
  
```

21. Dadas duas árvores AVL, dizemos que elas são **iguais** se armazenam os mesmos valores nas mesmas posições, isto é, possuem a mesma estrutura e os mesmos dados correspondentes. Implemente:

```

int avlIguais(NoArv* a1, NoArv* a2);
  
```

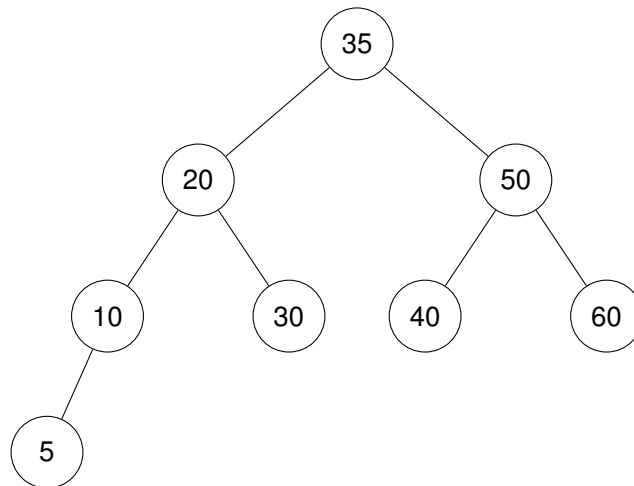
22. Considere a AVL abaixo.



Responda:

- (a) Quem é o irmão do nó 40?
- (b) Quais são os nós folha dessa árvore?
- (c) Qual é a altura da árvore?
- (d) Se o valor 70 for inserido, a árvore permanecerá balanceada ou exigirá rotação?

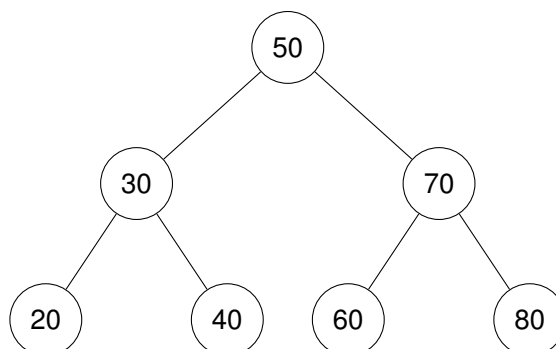
23. Considere a AVL abaixo.



Responda:

- (a) Qual é o menor valor armazenado na árvore?
- (b) Qual é o maior valor armazenado na árvore?
- (c) O nó 20 possui qual fator de balanceamento?
- (d) Se o valor 2 for inserido, a árvore exigirá rotação? Justifique brevemente.

24. Considere a AVL abaixo.



Responda:

- (a) Qual travessia em ordem essa árvore produz?
- (b) Qual é o sucessor em ordem do nó 60?
- (c) Se o valor 65 for inserido, ele ficará como filho esquerdo ou direito de qual nó?

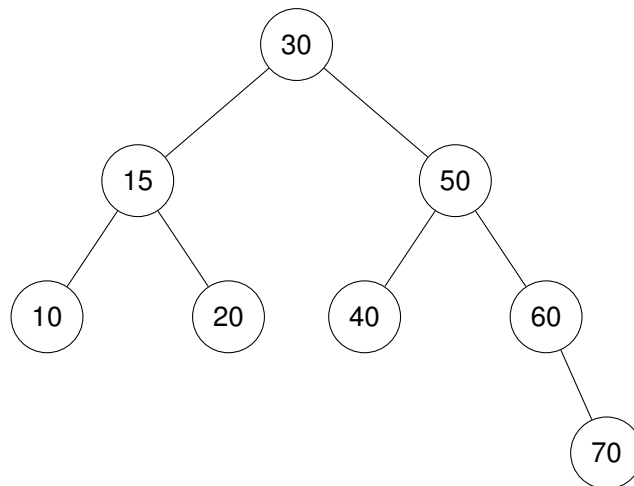
25. Considere uma AVL inicialmente vazia. Insira, nesta ordem, os valores:

60, 30, 80, 20, 40, 70, 90

Após construir a árvore, responda:

- (a) Se o valor 20 for removido, será necessária alguma rotação?
- (b) Se o valor 40 for removido logo depois, a subárvore esquerda da raiz ficará com qual altura?
- (c) A raiz continuará balanceada após essas remoções?

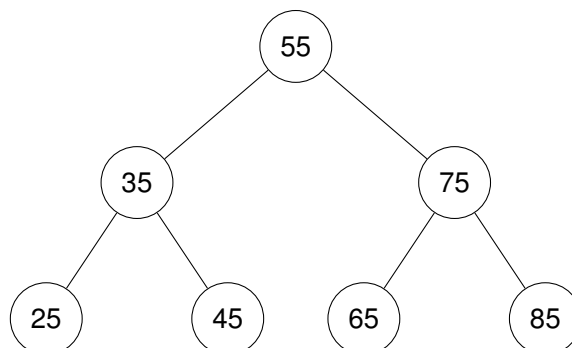
26. Considere a AVL abaixo.



Responda:

- (a) Qual é o nível do nó 20, considerando a raiz no nível 0?
- (b) Quantos nós internos a árvore possui?
- (c) Quantos nós folha a árvore possui?
- (d) Se o valor 45 for inserido, o fator de balanceamento do nó 50 passará a ser qual?

27. Considere a AVL abaixo.



Responda:

- (a) Se o valor 15 for inserido, qual nó ficará desbalanceado primeiro, se houver desbalanceamento?

- (b) Essa inserção exigirá rotação simples, dupla ou nenhuma?
- (c) Se o valor 95 for inserido em vez disso, haverá rotação?
- (d) Qual será o caminho percorrido na busca pelo valor 65?

28. Faça uma questão de análise de código: o aluno alterou a função abaixo.

```
NoArv* rotacaoDireitaEsquerda(NoArv* raiz) {  
    raiz->dir = rotacaoEsquerda(raiz->dir);  
    return rotacaoDireita(raiz);  
}
```

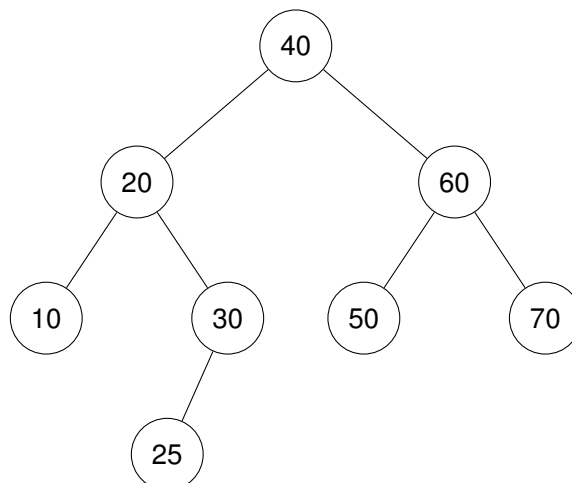
Explique qual é o erro lógico dessa implementação e reescreva a versão correta.

29. Um programador decidiu usar a função iterativa abaixo para inserir valores em uma AVL.

```
NoArv* inserir(Arvore* arvore, int valor) {  
    NoArv* novoNo = criarNo(valor);  
    if(arvore->raiz == NULL) {  
        arvore->raiz = novoNo;  
        return novoNo;  
    }  
    NoArv* atual = arvore->raiz;  
    NoArv* pai = NULL;  
    while(atual != NULL) {  
        pai = atual;  
        if(valor < atual->valor) atual = atual->esq;  
        else atual = atual->dir;  
    }  
    if(valor < pai->valor) pai->esq = novoNo;  
    else pai->dir = novoNo;  
    return novoNo;  
}
```

Explique por que essa função, sozinha, não garante que a árvore continue sendo AVL após a inserção.

30. Considere a AVL abaixo.



Assinale a alternativa correta.

- (a) Se o valor 27 for inserido, o primeiro nó desbalanceado será o 20, e será necessária uma **rotação dupla esquerda-direita**.
- (b) Se o valor 5 for inserido, o primeiro nó desbalanceado será o 40, e será necessária uma **rotação simples à direita**.
- (c) Se o valor 65 for inserido, a árvore deixará de ser AVL e será necessária uma **rotação simples à esquerda na raiz**.
- (d) Se o valor 55 for inserido, nenhuma rotação será necessária, e o fator de balanceamento do nó 60 passará a ser 1.
- (e) Se o valor 25 for removido, o nó 30 ficará com fator de balanceamento igual a -1 , e a árvore continuará AVL sem rotação.